



构造函数

▼ 构造函数

类必须定义与类名相同的构造函数。

析构函数会在构造函数之前加一个“~”，该函数会在某个实例被销毁时执行。

```
class Test{  
private:  
    int data;  
  
public:  
    Test(int a):data(a){}  
    ~Test(){  
        cout<<"销毁了一个"<<typeid(*this).name()<<"类型的对象"<<endl;  
    }  
};
```

C++

类的构造函数和析构函数可以在类内定义，此时会被视为内联函数。因为类内部定义的所有非虚函数都会被编译器隐式地视为 `inline`，包括构造函数。

但是否真正内联展开，仍然取决于编译器的优化策略。

在类外定义内联函数，则需要在定义处显式使用 `inline` 修饰符。

如果类的定义在头文件中，此时类内定义的函数必须是内联函数，因为类内定义的函数如果为非内联函数，将会触发重定义的错误。

并且由于内联是否展开取决于编译器，即使显式定义的内联函数，也不一定会被展开，所以不建议在头文件中定义较为复杂的函数。

如果需要将类定义为字面类型，即在编译期即可使用的类型，则需要至少有一个 `constexpr` 修饰的构造函数。除此之外，还需要满足所有非静态成员都必须是字面类型、没有虚函数或虚基类、析构函数必须是 `constexpr`（C++20 起默认满足）。

字面类型的类可以作为模板的非类型参数，并将该类型的 `constexpr` 编译期常量对象作为模板的参数。

▼ 初始化列表

出现在构造函数的冒号后面，在进入函数体之前进行初始化。

理解该概念需要区分初始化与赋值，比如 `int a=0;` 和 `int a;a=0;` 之间的区别。

尤其是对于 `const` 类型与引用类型，这些类型只能在初始化时确定其值，初始化后不能再进行赋值，所以必须在进入函数体之前进行列表初始化，而不能在函数体中进行赋值。

```
class A {
public:
    int& ref;    // 引用成员
    A(int& r) : ref(r) {} //正确
    A(int& r) { ref = r; } //错误
};
```

C++

▼ 隐式构造

指编译器自动调用单参数构造函数来进行类型转换，无需手动显式调用构造函数。

要触发隐式构造，需要满足以下两个条件：

1. 类必须有单参数构造函数（或所有参数除了第一个都有默认值）。
2. 构造函数不能被 `explicit` 修饰（否则会阻止隐式转换）。

满足以上条件时，当在某种场景下，需要接受 A 类，但却接受了 B 类，此时编译器会尝试将 B 类作为 A 类构造函数的参数，生成一个 A 类的对象。例如，

```
class A{
public:
    int a;
    A(int i):a(i){}
};
A test = 1;
```

C++

该例中，`A test = 1;` 的初始化并不是对 `=` 进行了重载，而是触发了隐式构造。`int` 类型的数值与 A 类不匹配，所以编译器会尝试将 `int` 数据作为 A 类构造函数的参数，生成一个 A 类的对象并返回。

如果接下来直接输出对象 a，会发现并不能直接将 a 作为 `int` 类型输出，这是因为隐式构造只是在初始化时被调用，后续的访问需要使用的隐式类型转换。

如果接下来想通过 `=` 重新对 a 赋值，也是不可行的，同样是因为隐式构造只能调用在初始化时，如果想通过 `=` 重新赋值，需要对 `=` 进行重载。

`std::reference_wrapper` 的使用中也用到了隐式构造：

C++

```
MyClass obj(10);
std::vector<std::reference_wrapper<MyClass>> vec;
vec.push_back(obj); // 隐式构造 std::reference_wrapper<MyClass>(obj)
```

`vec` 期待的变量类型是 `std::reference_wrapper<MyClass>`，而实际传入的类型是 `MyClass`，在这个过程中就触发了 `MyClass` 到 `std::reference_wrapper<MyClass>` 的隐式构造。

▼ 拷贝初始化

在 C++11 之前，大部分初始化的方法是：`Myclass a = Myclass();`。

这种方法使用拷贝初始化，先创建一个临时对象 `Myclass()`，然后用它来初始化 `a`。

C++11 开始，引入了列表初始化，可以使用花括号进行初始化，如 `Myclass a = Myclass{};`，这种初始化叫做拷贝列表初始化，能避免一些隐式转换问题，更安全。

不过该方法属于列表初始化 + 拷贝初始化，先使用列表初始化创建一个临时对象再进行拷贝。

`Myclass a = Myclass();` 与 `Myclass a = Myclass{};` 在不支持列表初始化的类中，均通过普通构造函数初始化后再进行拷贝；在支持列表初始化的类中，则取决于列表构造函数的具体内容是否与普通构造函数相同。

不过现代 C++ 编译器通常会做返回值优化（RVO）或拷贝消除，所以实际不会调用拷贝构造函数，而是直接在 `a` 的存储上构造对象。

▼ 直接初始化

形如 `Myclass a();` 的初始化被认为是直接初始化，在 `a` 变量的地址上直接进行构造。

直接初始化不属于拷贝初始化，所以也不受 `explicit` 限制，`explicit` 只限制拷贝初始化。

但是这种构造方式有时会被认为是一个函数声明（C++ 的最著名的解析歧义——最令人头疼的“Most Vexing Parse”）。

当编译器看到 `Type name(Type())`，它可能理解为：

- 返回类型是 `Type`
- 函数名是 `name`
- 参数是函数指针 `Type (*)()`

此时极大可能会触发 Most Vexing Parse。

这种情况往往出现在：

- 单参数构造，参数是 `类型名 + ()`
- 多参数构造，且参数中全部为 `类型名 + ()`

虽然列表初始化也属于直接初始化，但是不建议将 `Myclass a();` 与 `Myclass a{};` 当作等效，不过为了避免 Most Vexing Parse，可以使用列表初始化。

在明确了解并可以避免 Most Vexing Parse，同时类不存在 `initializer_list` 构造函数，此时 `Myclass a();` 与 `Myclass a{};` 确实可以等效使用。

▼ 列表初始化

相比于拷贝初始化与直接初始化，`Myclass a{};` 的写法是更加推荐和简洁的，该方法为直接列表初始化，`Myclass a = Myclass{};` 则是拷贝列表初始化。

列表初始化的本质是 `Myclass({...})`，将 `{}` 中的内容作为参数构造

`std::initializer_list<T>`，并调用可以接受 `std::initializer_list<T>` 类型的构造函数，并在语法中省略外层括号。

直接列表初始化属于直接初始化的一种形式，当类有匹配的 `initializer_list` 构造函数时，列表初始化会优先调用它；

如果没有 `initializer_list` 构造函数，列表初始化仍然可以调用普通构造函数或执行聚合初始化。

- 在没有 `std::initializer_list<T>` 类型的构造函数时，列表初始化会使用列表中的参数匹配普通构造函数；
- 在存在 `std::initializer_list<T>` 类型的构造函数时，列表初始化会优先使用该构造函数。

`std::initializer_list<T>` 参数的优势在于接受不定数量的同类型参数，在此情况以外，不建议实现该类型的构造函数，否则不利于其他类型构造函数的重载，因为会在列表初始化时优先匹配到该构造函数。

`std::initializer_list<T>` 参数一般不会直接使用，可以使用 `std::vector<T>` 等类型接受该参数，从而触发隐式构造。

在实现 `std::initializer_list<T>` 参数的构造函数之后，需要使用普通构造函数时：

- 如果构造中参数类型不同，则会不匹配 `std::initializer_list<T>`，从而匹配普通构造函数；
- 如果构造中参数类型相同，则会匹配至 `std::initializer_list<T>` 类型的构造函数，无法匹配至普通构造函数。此时应避免使用列表初始化，而是使用 `()` 的拷贝初始化。

如果类是聚合类型（简单结构体，没有用户定义构造函数），则可以直接用 `{}` 对成员按顺序初始化，这不是依赖 `initializer_list` 构造函数。

▼ 同类实例构造

通过列表初始化可以为一个类构造一个实例，除此之外，列表初始化还允许通过一个类的一个实例构造一个新的实例。

```
C++

struct Point {
    int x = 0;
    int y = 0;
};

inline constexpr Point origin{}; // 一个具名常量对象

int main() {
    auto a = origin; // 拷贝
    auto b = origin{}; // ✅ 构造新的 Point 对象，等价于 Point{}
}
```

其本质是将 `origin{}` 转换为了 `decltype(origin){}`，所以

- `obj{}` 实际等价于 `decltype(obj){}`
- 该写法仅当以下条件成立时可用：
 - `decltype(obj)` 是非引用的完整类型
 - `decltype(obj)` 是类类型，具有可访问的默认构造函数
- 以下情况不可使用：
 - `obj` 是基本类型变量（如 `int`，`double`），基本类型没有 `{}` 语法意义
 - `obj` 是引用类型（如 `T&`，`const T&`），`decltype(obj)` 是引用类型，不能构造引用类型对象
 - 类型不可默认构造，比如显式删除默认构造函数：`Foo() = delete;`
- 若需在模板中使用：
 - 限定该模板接受的类型不是基本类型
 - 移除引用修饰：`std::remove_reference_t<decltype(obj)>{}`